

The background is a light cream color, decorated with various blue geometric shapes. There are several solid blue circles of different sizes. Some circles are surrounded by a cluster of small blue dots. There are also thin blue concentric circles and rings scattered throughout the design.

Apresentação

Caminhão

Objetivos

O projeto tem como objetivo simular um sistema de fiscalização de velocidade utilizando uma Raspberry Pi 3B+ e aplicações de IoT.

- Utilizar uma Raspberry Pi 3B+.
- Verificar a velocidade do veículo.
- Comparar a velocidade com o limite da via.
- Identificar quando o caminhão ultrapassa 80 km/h.
- Utilizar Docker, MQTT e Node-RED para realizar o sistema.

Art.61 CTB

Ordem	Tipo de Via	Limite em Km/h
1	Rodovia	80/90
2	Local	30
3	Arterial	60
4	Coletora	40

Processo do projeto

Começamos preparando um nó compartilhado:

- Utilizamos o Raspberry Pi Imager.
- Escolhemos o Raspberry Pi 3B+.
- Utilizamos o Raspberry Pi OS Lite 64-bit — Legacy.
- Configuramos o Raspberry Pi rpi-106. X
- Configuramos Wi-Fi, usuário e acesso por SSH.
- Expandimos o Filesystem para utilizar todo o cartão.
- Reiniciamos e atualizamos os pacotes. X
- Instalamos e configuramos o Docker. X

Falhas

Configurando o docker

```
services:
  portainer:
    image: portainer/portainer-ce:latest
    container_name: portainer
    restart: always
    command: --admin-password '$$2b$$10$$7nPyOAsha7uiMAeA55/6Z.muRIi5.jf8G7JPteVN90tdIDuSS6vS2 '
    ports:
      - "8000:8000"
      - "9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data

volumes:
  portainer_data:
```

Subindo o docker

```
1 cd ~/fiscalizacao/portainer && docker compose up -d
2 cd ~/fiscalizacao/node-red && docker compose up -d
```

Processo do projeto

Depois começamos a configurar as aplicações:

- Criamos uma pasta para o projeto.
- Criamos uma subpasta para cada serviço.
- Utilizamos o Docker Compose para organizar os serviços.
- Configuramos o Portainer para administrar os containers.
- Subimos o Node-RED.
- Configuramos o Mosquitto, que funciona como broker MQTT. X
- Criamos uma rede compartilhada para o Node-RED e o MQTT. X

Falhas

Configurando o MQTT

```
services:
  mosquitto:
    image: eclipse-mosquitto:latest
    container_name: mosquitto
    restart: always
    ports:
      - "1883:1883"
    volumes:
      - ./config:/mosquitto/config
      - mosquitto_data:/mosquitto/data
      - mosquitto_log:/mosquitto/log
    networks:
      - fiscalizacao

volumes:
  mosquitto_data:
  mosquitto_log:

networks:
  fiscalizacao:
    external: true
```

Subindo o MQTT

```
cd ~/fiscalizacao/mqtt
docker compose up -d
docker ps --filter name=mosquitto
```

Aplicações

- 1. Raspberry Pi: É o computador onde toda a infraestrutura é executada.
- 2. Docker: Executa as aplicações em containers.
- 3. Docker Compose: Organiza e facilita a configuração dos containers.
- 4. Portainer: Permite administrar os containers por uma interface gráfica.
- 5. MQTT Broker — Mosquitto: Gerencia a troca de mensagens entre os dispositivos.
- 6. Node-RED: Permite criar visualmente o fluxo de processamento e comunicação.

Conclusão

- Utilizamos a Raspberry Pi para executar as aplicações através do Docker.
- Organizamos cada serviço em seu próprio container.
- Mas cometemos erros na configuração do docker e do mosquitto, e dificuldades em criar uma rede compartilhada para permitir a comunicação entre o Node-RED e o MQTT. Configurar o Mosquitto como broker.

Conclusão e considerações

- Dificuldade na compreensão do projeto
- Configuração simples e fácil
- Ritmo acelerado → Dificuldade
- No fim, paramos na parte de testes de fiscalização do NODE-RED



Obrigada

Amal, Livia e Vinicius